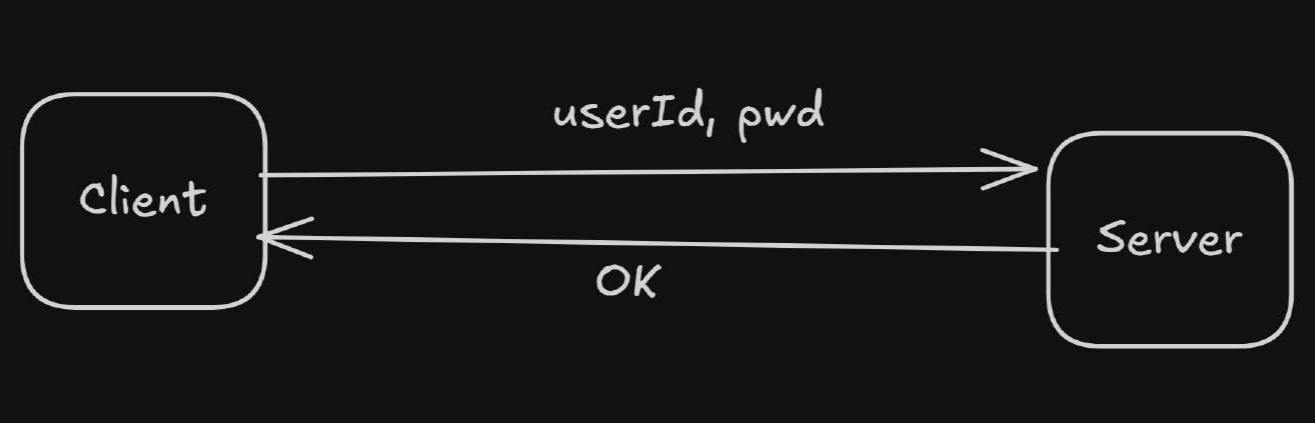
Session-Based Auth



What is a Session?

A temporary server-side storage that keeps track of a user's identity and state across multiple HTTP requests.

Why do we need sessions?

HTTP is stateless → every request is independent.

Request 1 → Server (no memory)
Request 2 → Server (no memory)

Server cannot remember:

who you are
whether you logged in

Naive Solution (Why Sessions Exist)

Send credentials on every request

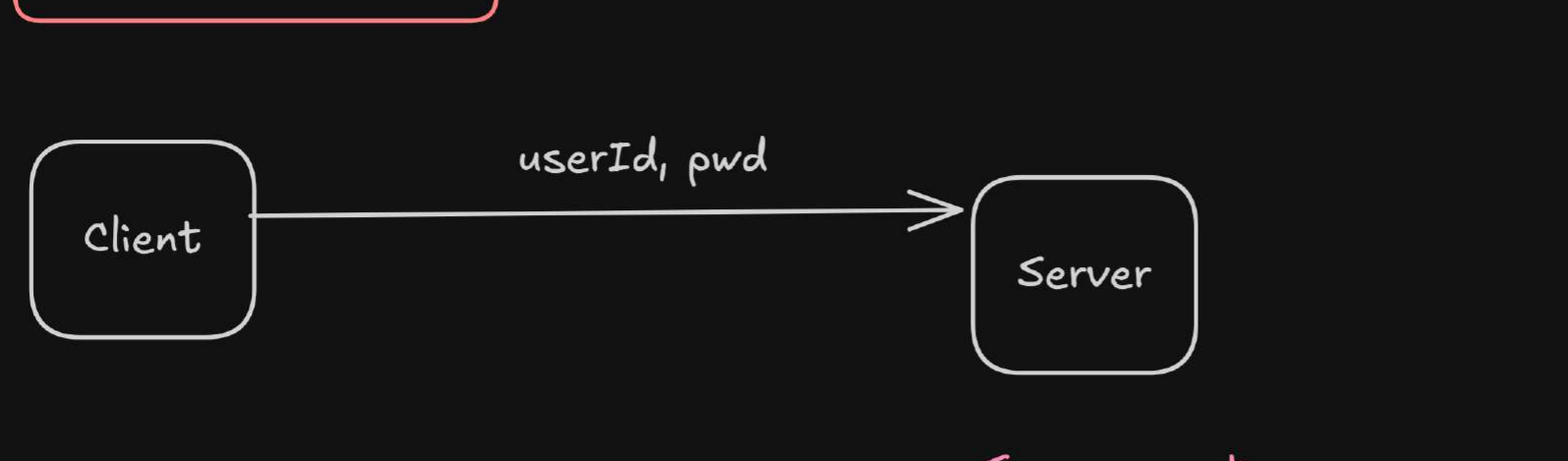
Problems:

-> insecure
-> heavy
-> bad UX

So we need:

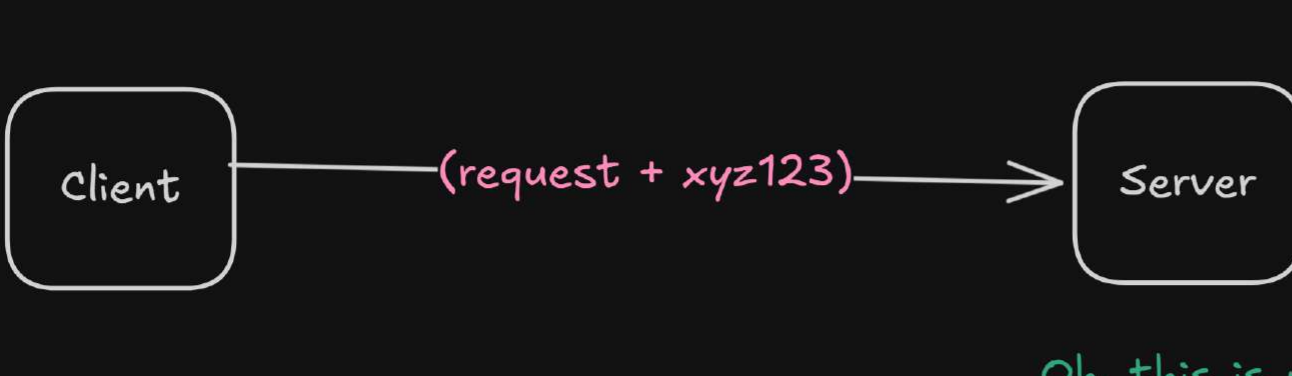
"Login once → remember user"

Session solves this



Server creates:
session_id = "xyz123"

Server stores:
xyz123 → user_id = 42



Oh, this is user 42

Session Store (Server Side)

session_id: xyz123
----
user_id: 42
roles: [ADMIN]
created_at: 10:00
last_active: 10:05

Session = data + identity mapping

What is a Cookie?

A small piece of data stored in the browser, sent automatically with every request.

Key Point

-> Cookie does NOT store user data (in session auth)
-> It only stores session_id

Cookie Example

```
Set-Cookie: session_id=xyz123;
            HttpOnly;
            Secure;
            SameSite=Lax;
```

Cookie

session_id = xyz123

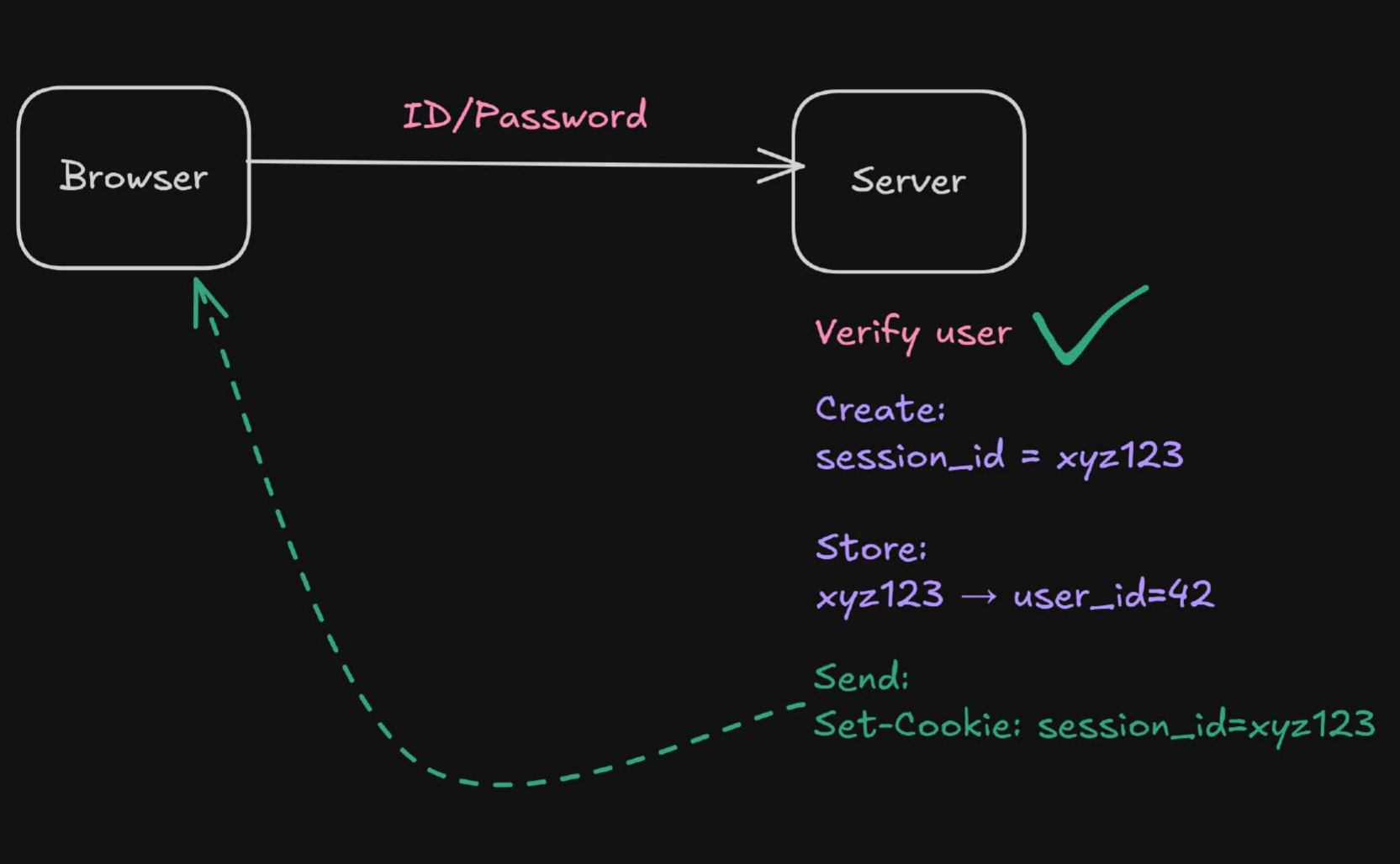
Session Store

xyz123 → user_id = 42

Cookie = pointer
Session = actual data

Full Session-Based Auth Flow

Step 1: Login



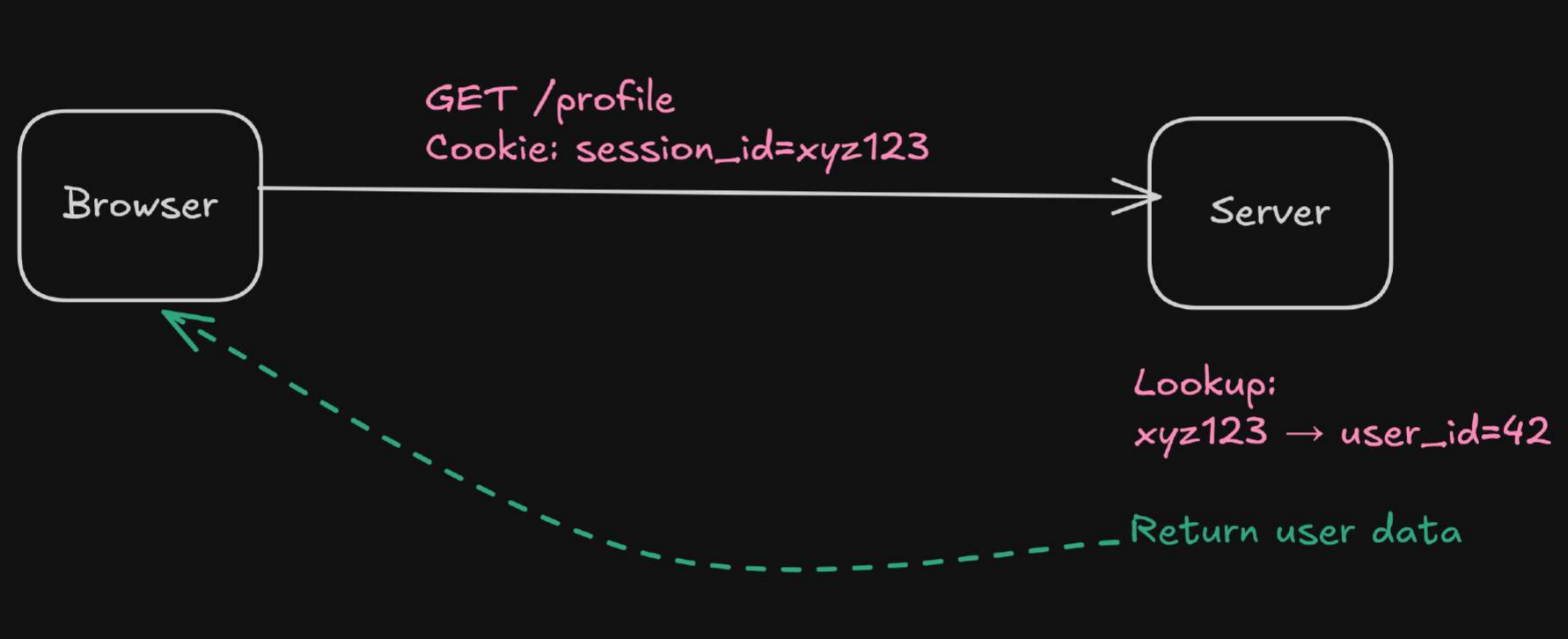
Verify user ✓

Create:
session_id = xyz123

Store:
xyz123 → user_id=42

Send:
Set-Cookie: session_id=xyz123

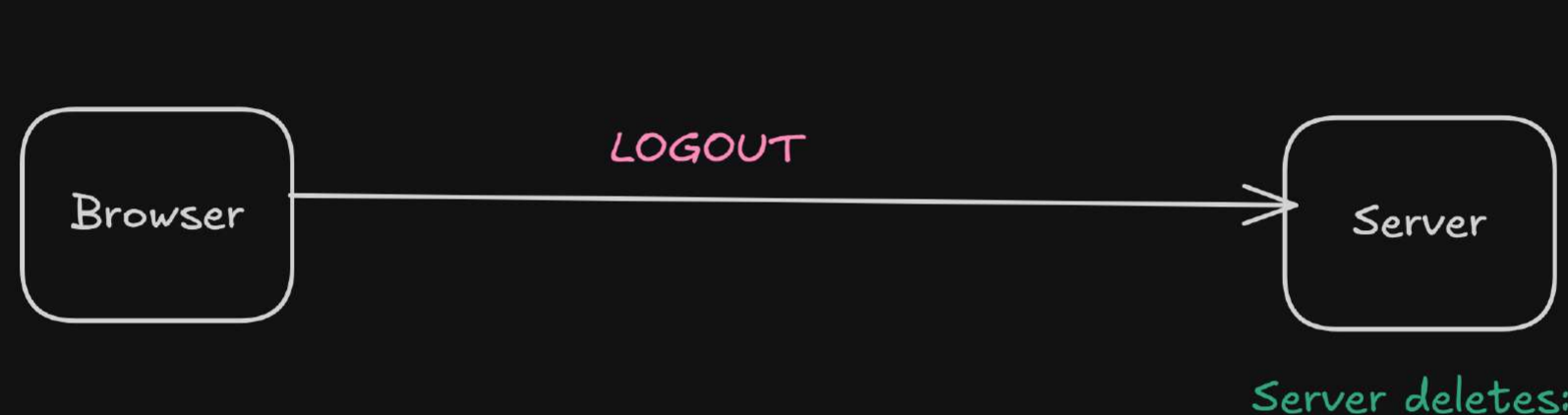
Step 2: Subsequent Requests



Lookup:
xyz123 → user_id=42

Return user data

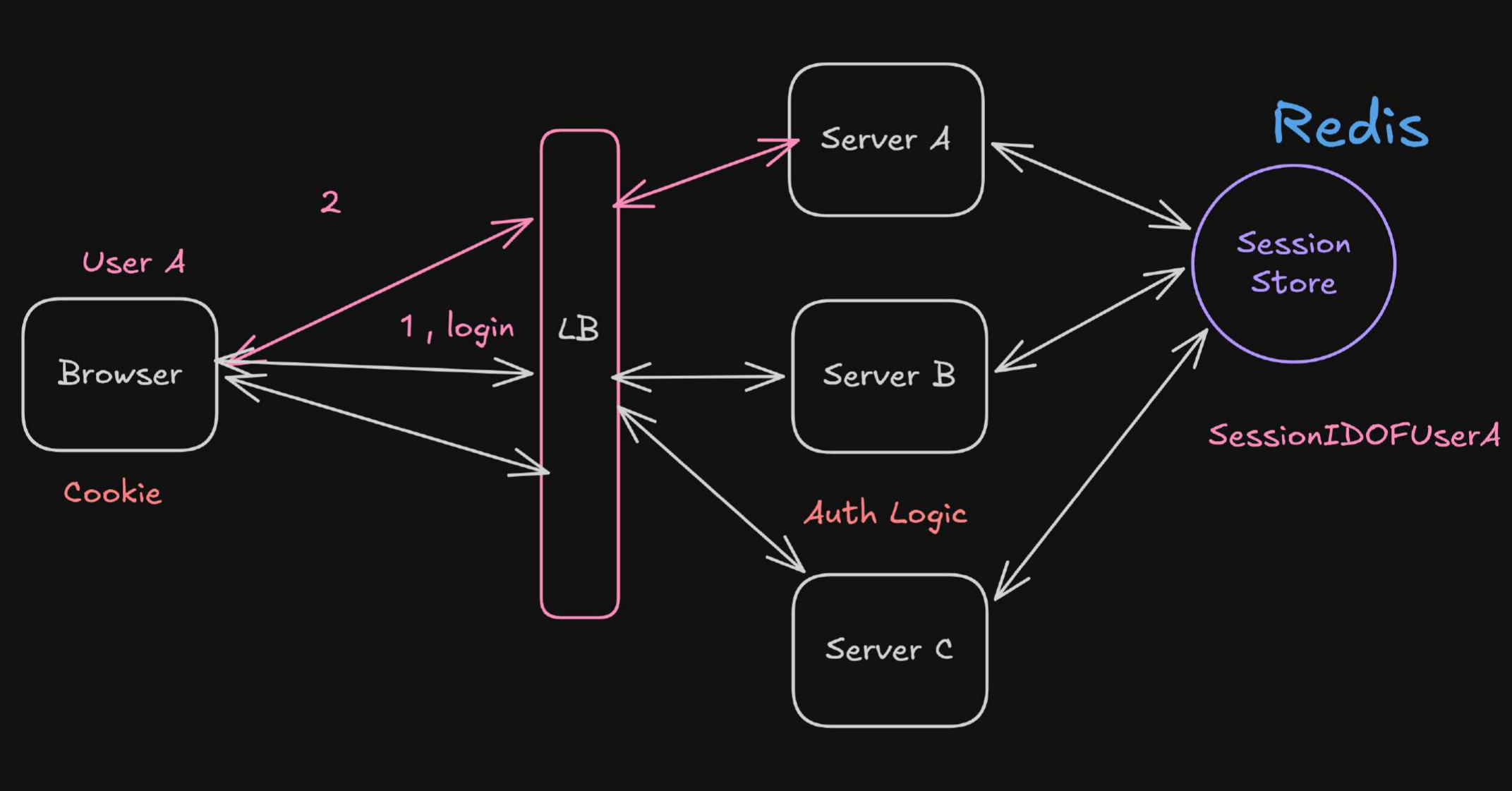
Step 3: Logout



Server deletes:
xyz123

Now cookie becomes useless

High-Level Architecture (HLD)



Scaling Problem

User → Server A (session exists)

Next request:
User → Server B (no session)

Solution 1: Sticky Sessions

Load Balancer → always send user to Server A

Bad:

not fault tolerant
not scalable

Solution 2: Shared Session Store (Industry Standard)

Any server can authenticate user
Horizontally scalable

Why Redis (Not DB)

Login → Write once
Requests → Read many times
Logout → Delete

Read-heavy system

and also, we need low latency

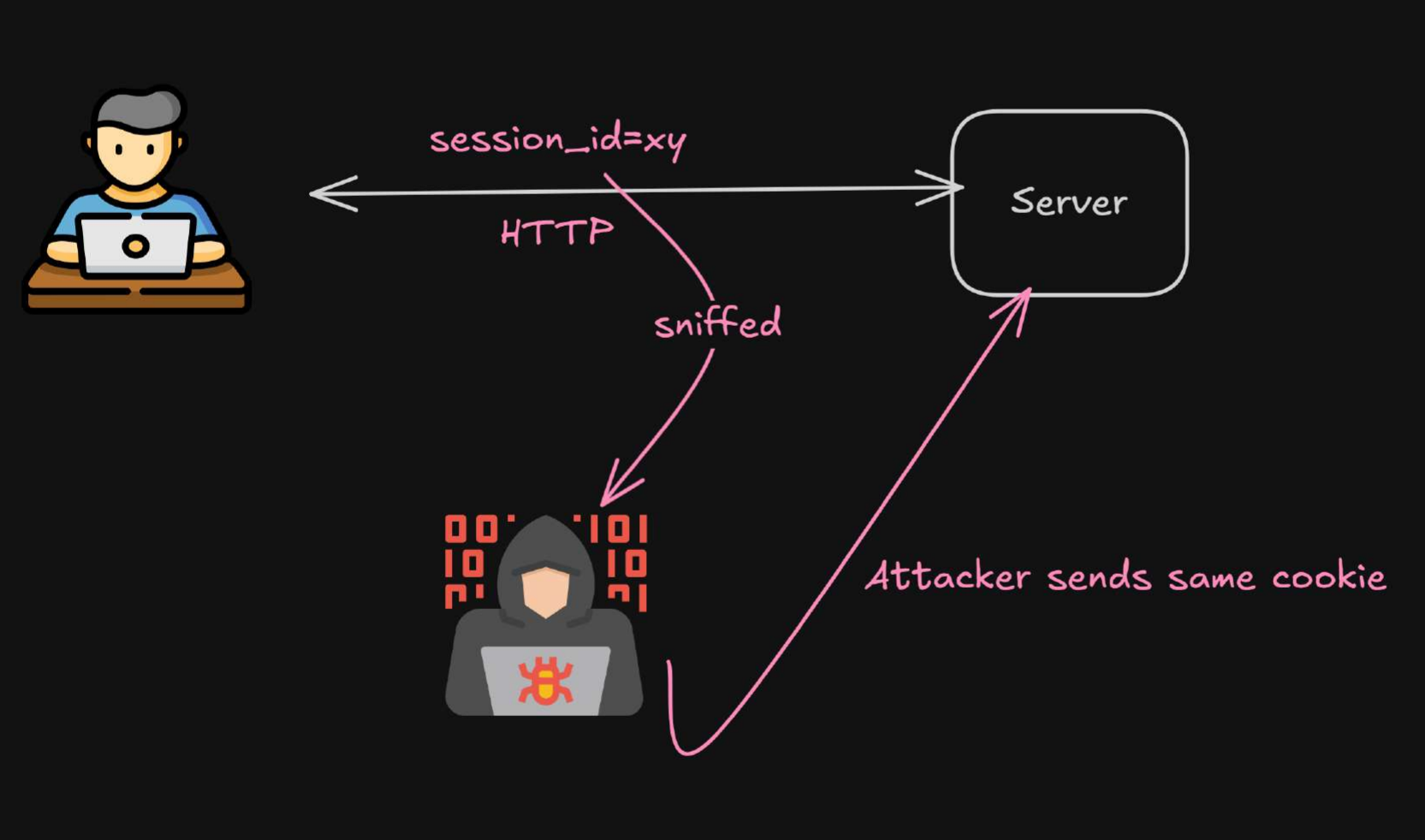
Security Design

1. Session Hijacking

Attacker steals session_id and uses it to impersonate user

How Attack Happens

Case 1: Network Sniffing (HTTP)



Case 2: XSS (Stealing via JS)

Malicious JS:
document.cookie → send to attacker

Impact:

Full account takeover
No password needed

Solutions

1. Secure Flag

Secure

-> Cookie only sent over HTTPS

2. HttpOnly Cookie

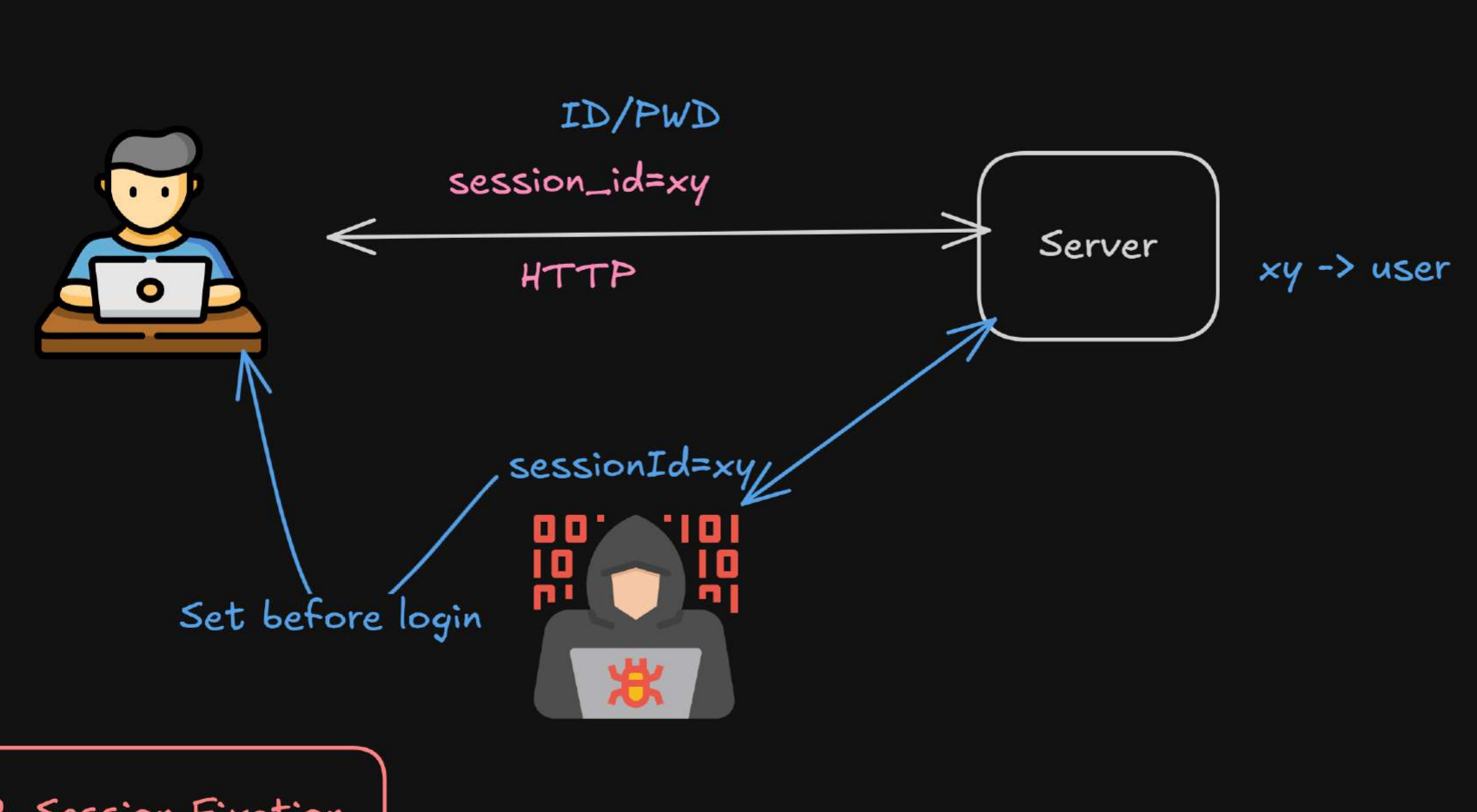
Set-Cookie: session_id=xyz;
HttpOnly

-> JS cannot access cookie → prevents XSS theft

3. Short Session TTL

session expires quickly

-> reduces attack window



2. Session Fixation

Attacker sets session_id BEFORE login

Attacker creates:
session_id = abc123

Victim logs in using same session

Server:
abc123 → user_id=42

Attacker already knows abc123

Solution

-> Regenerate Session ID AFTER Login
Before login:
abc123

After login:
xyz999 (NEW)

-> Old session destroyed

-> Rule
Never trust session_id before authentication

3. CSRF (Cross-Site Request Forgery)

Attacker tricks browser into sending request using existing session

User logged in to bank.com

User visits attacker.com

Attacker triggers:
POST /transfer

Browser auto sends:
Cookie: session_id=xyz123

Solutions

1. SameSite Cookie
SameSite=Lax / Strict

-> Prevents cross-site requests

2. CSRF Token (Standard)

Client sends:
csrf_token=abc

Server validates

Server → send CSRF token
Client → sends token in request
Server → verify token

4. Session Replay Attack

5. Session Prediction (Weak IDs)

When to use Session-Based Authentication

-> Server-rendered web apps (SSR)
(e.g., Django, Spring Boot)
-> Monolith architectures
Single backend controlling everything
-> Web apps with browsers only
Cookies work naturally
-> Need easy logout / session invalidation
Server can instantly kill session
-> High security control required
Sessions stored server-side (can track, revoke)
-> Small to medium scale apps
Simpler to implement

When NOT to use Session-Based Auth

-> Microservices architecture
Hard to share sessions across services
-> Mobile apps / APIs
Cookies unreliable → prefer tokens
-> Stateless systems required
Sessions = stateful (needs Redis/DB)
-> High scalability systems
Session storage becomes bottleneck
-> Cross-domain / third-party APIs
Cookies don't work well

DEMO